RescueOps

Problem Statement + What to build in agent.py

PART A: Problem Statement

1. Background

During a disaster, rescue teams receive several calls for help at the same time. Each call (mission) has a different **severity**, **distance** and **deadline**. Roads may be blocked, the weather may be bad, and the team has only a **limited amount of fuel, medical kits and personnel**. A wrong decision, such as sending a team on a blocked road or without enough fuel, can cost lives.

2. Objective

Build an **AI agent** (in Python, using LangGraph and LangChain) that reads a rescue scenario and returns **one safe and smart decision**: which mission to send help to, and how, or whether to wait, reject or ask for resupply. The competition has **5 rounds** of increasing difficulty (100 points in total). A **single agent.py** must handle all five rounds, and each round includes everything from the earlier ones.

Round	Title	Points	New challenge
1	Basic Rescue	10	One mission; check fuel and deadline
2	Blocked Routes	15	Blocked roads and alternate routes
3	Limited Resources	20	Fuel, kits and personnel are limited; resupply
4	Critical Priorities	25	Many missions with tight deadlines; severity vs feasibility
5	Adaptive RescueOps	30	Weather changes and free-text updates; decide again

3. Input: the Scenario (case)

```
case_id, round, stage
weather ("clear" | "rain" | "storm")
resupply_possible (true | false)
resources: {fuel, kits, personnel}
missions: [ {id, severity (1-5), distance, deadline, kits, personnel,
             route_blocked, alternate_distance | null} ]
update_notice, update_report   (Round 5 only, from stage 1 onwards)
```

4. Output: the Decision

```
{ "action": "dispatch | reroute | wait | reject | resupply",
  "priority": 1-5,
  "reason": "short explanation",
  "mission_id": "M1",            (needed if more than one mission)
  "uses_alternate_route": true/false,
  "fuel_needed": n, "kits_needed": n, "personnel_needed": n }
```

Required fields: action, priority (integer 1-5), reason (non-empty). The rest are optional.

Action	Meaning
dispatch	Send a team to the mission by the direct route
reroute	Send a team by the alternate route (direct road is blocked)
wait	Nothing can be done safely right now, but the situation may improve
reject	The mission cannot be completed at all (for example, no route or the deadline cannot be met)
resupply	Request more fuel, kits or personnel because resources are the problem

5. Rules common to all rounds

Rule	Detail
Fuel needed	ceil(route distance / 10)
Travel time	ceil(route distance x weather factor). Must be less than or equal to the mission deadline
Weather factor	clear = 1.0, rain = 1.25, storm = 1.5
Route distance	The direct distance if the road is open; the alternate_distance if the direct road is blocked
Blocked road	A blocked direct route must never be used. If alternate_distance is null, there is no route and the mission is infeasible
Resources	The team needs fuel >= fuel needed, kits >= mission kits and personnel >= mission personnel
One decision	Each decision concerns exactly one mission
Safety first	An unsafe dispatch (blocked road, not enough resources or a missed deadline) is never acceptable, whatever the severity
Choosing between missions	Among feasible missions, prefer the higher severity. Break ties by the tighter deadline, then by lower fuel use

Feasibility check (used in every round)

```
km   = distance if not route_blocked else alternate_distance   # None -> infeasible
fuel = ceil(km / 10)
time = ceil(km * weather_factor)
feasible = (km is not None) and fuel <= resources.fuel
           and mission.kits <= resources.kits
           and mission.personnel <= resources.personnel
           and time <= mission.deadline
```

Round-by-round Problem

Round 1: Basic Rescue (10 points)

Description: A single mission, clear weather and an open road. The agent only has to decide whether the rescue is possible with the available fuel and in the available time.

Input:

- One mission; weather is always clear; route_blocked is false.
- Resources: fuel, kits, personnel.

Task:

- Compute fuel needed and travel time for the mission.
- Dispatch if fuel, kits, personnel and deadline are all satisfied.
- Otherwise reject or wait, and give the reason.

Constraints:

- Exactly one mission per case.
- Decision must contain action, priority (1-5) and a reason.

Sample:

```
Sample 1
weather: clear | resources: fuel 5, kits 3, personnel 2
M1: severity 4, distance 30, deadline 40, kits 1, personnel 1
fuel = ceil(30/10) = 3 <= 5     time = ceil(30 x 1.0) = 30 <= 40
Output: action = dispatch, priority = 4, mission_id = M1, fuel_needed = 3

Sample 2
weather: clear | resources: fuel 5, kits 3, personnel 2
M1: severity 5, distance 80, deadline 100, kits 1, personnel 1
fuel = ceil(80/10) = 8 > 5  -> not enough fuel
Output: action = reject (or wait), priority = 5, reason = "Insufficient fuel"
```

Round 2: Blocked Routes (15 points)

Description: One or two missions. Some direct roads are blocked. A longer alternate route may exist, but it needs more fuel and more time, and sometimes it does not exist at all.

Input:

- 1-2 missions with route_blocked and alternate_distance (a number or null).
- Weather may be clear or rain.

Task:

- Never use a blocked road.
- If blocked and an alternate exists, run the feasibility check on the alternate distance and return reroute (or uses_alternate_route = true).
- If blocked and the alternate is null, the mission is infeasible.
- With two missions, choose the feasible one with higher severity.

Constraints:

- Fuel must be calculated from the route actually used (alternate distance for a rerouted mission).
- mission_id is required when there is more than one mission.

Sample:

```
Sample 1
weather: clear | resources: fuel 6, kits 2, personnel 2
M1: severity 5, distance 20 (BLOCKED), alternate 35, deadline 40, kits 1, personnel 1
fuel = ceil(35/10) = 4 <= 6     time = 35 <= 40
Output: action = reroute, mission_id = M1, uses_alternate_route = true, fuel_needed = 4

Sample 2
M1: severity 5, distance 20 (BLOCKED), alternate = null
No usable route.
Output: action = reject, priority = 5, reason = "Route blocked and no alternate"
```

Round 3: Limited Resources (20 points)

Description: Two or three missions, and the team's fuel, kits and personnel are limited. Sometimes no mission can be served with what is available. Resupply may or may not be possible.

Input:

- 2-3 missions; limited resources; resupply_possible may be true.

Task:

- Check every mission against the resources, route and deadline.
- Dispatch (or reroute) the best feasible mission.
- If nothing is feasible: choose **resupply** when resupply is possible and the shortage is only in resources; choose **reject** when the route or deadline makes it impossible anyway; otherwise **wait**.

Constraints:

- Never dispatch a mission the resources cannot support, even if its severity is the highest.
- Doing nothing while a feasible rescue exists is a poor decision.

Sample:

```
Sample 1
weather: clear | resources: fuel 4, kits 2, personnel 2
M1: severity 5, distance 30, deadline 40, kits 3, personnel 1 -> kits 3 > 2, infeasible
M2: severity 3, distance 25, deadline 40, kits 1, personnel 1 -> fuel 3, time 25: feasible
Output: action = dispatch, mission_id = M2, priority = 3

Sample 2
resupply_possible: true | resources: fuel 1, kits 2, personnel 2
M1: severity 4, distance 30, deadline 40 -> needs fuel 3 > 1, but the deadline is fine
Output: action = resupply, priority = 4, reason = "Fuel shortage, resupply available"
```

Round 4: Critical Priorities (25 points)

Description: Three to five missions with tight deadlines compete for the same limited resources. The most severe mission is not always the one that can be saved, so the agent must balance severity and feasibility.

Input:

- 3-5 missions; tight deadlines; limited resources; blocked roads and rain are possible.

Task:

- Filter out every infeasible mission first (safety first).
- From the rest, choose the highest severity; break ties by the tighter deadline, then lower fuel use.
- Return dispatch or reroute for that mission, with priority equal to its severity.
- If none is feasible, fall back to resupply / wait / reject as in Round 3.

Constraints:

- Exactly one mission is chosen per decision.

- Always output a consistent, deterministic result for the same input.

Sample:

```
weather: clear | resources: fuel 6, kits 4, personnel 3
M1: severity 5, distance 90, deadline 120 -> fuel 9 > 6   infeasible
M2: severity 4, distance 50, deadline 45  -> time 50 > 45 infeasible
M3: severity 4, distance 40, deadline 50  -> fuel 4, time 40  FEASIBLE
M4: severity 2, distance 10, deadline 30  -> fuel 1, time 10  FEASIBLE
Output: action = dispatch, mission_id = M3, priority = 4, fuel_needed = 4
(The highest severity, M1, is skipped because it cannot be done safely;
 M3 wins over M4 because it has the higher severity.)
```

Round 5: Adaptive RescueOps (30 points)

Description: The full problem, now changing over time. The case arrives in several stages. After the first stage, the weather may change and a free-text update report describes what changed (new weather, blocked roads, resource changes). The fields of the original case become outdated, and **the report is the truth**.

Input:

- Everything from Rounds 1-4, plus stage, update_notice and update_report (free text).
- Example report: "Weather is now storm. Route to M2 is now blocked. Fuel is now 4."

Task:

- Read the update report and extract the changes (weather, route status, resources).
- Apply them to the scenario (an LLM can be used to interpret the text, but its result must be checked).
- Decide again on the updated scenario using the Round 4 logic.
- Each stage is a separate call: every decision must reflect the current state only.

Constraints:

- Round 1-4 logic must still work inside this round.
- An LLM reading of the report must be double-checked (blocked route, resources, deadline) before it is trusted.
- An invalid or unsafe decision at any stage is not acceptable.

Sample:

```
Stage 0 (original case)
weather: clear | resources: fuel 8, kits 3, personnel 2
M1: severity 3, distance 20, deadline 35, kits 1, personnel 1
M2: severity 5, distance 15, alternate 40, deadline 50, kits 1, personnel 1
Best choice: M2 (severity 5, fuel 2, time 15)

Stage 1 update_report:
"Weather is now storm. Route to M2 is now blocked. Fuel is now 4."
Updated state: weather storm (x1.5), fuel 4, M2 direct road blocked
M2 via alternate: time = ceil(40 x 1.5) = 60 > 50 -> infeasible
M1: fuel 2 <= 4, time = ceil(20 x 1.5) = 30 <= 35 -> feasible
Output: action = dispatch, mission_id = M1, priority = 3, fuel_needed = 2
(The decision changes because the situation changed: the severe mission M2
 is no longer safely reachable, so the agent saves M1.)
```

PART B: What to do in agent.py

Structure of agent.py

The file has **5 independent solvers** (solve_round1 to solve_round5), each with its own helper functions. **choose_rescue(case)** is the evaluator's entry point; it reads **case["round"]** and calls the matching solver (already written for you). Keep each round's solver separate so that changing one never breaks another.

Round	Helper functions in the skeleton	What you need to write
1	calculate_fuel, calculate_travel_time, assess_mission, solve_round1	Compute fuel and time; dispatch if feasible, otherwise reject, with a reason
2	analyse_routes, compare_routes, select_best_route, solve_round2	Use the alternate route when blocked; reroute or reject
3	calculate_resource_demand, evaluate_resource_availability, allocate_resources, solve_round3	Resource checks; resupply / reject / wait rule when nothing is feasible
4	evaluate_severity, calculate_urgency, rank_emergencies, solve_round4	Drop infeasible missions, then pick the best by severity, deadline, fuel
5	call_ollama, understand_report, extract_key_information, generate_adaptive_decision, solve_round5	Read the report (Ollama), verify it, update the case, run Round 4 logic

Step 0: Common feasibility function (write this first)

```
import math
WEATHER = {"clear": 1.0, "rain": 1.25, "storm": 1.5}

def check_mission(m, resources, weather):
    if m["route_blocked"]:
        km = m.get("alternate_distance")     # may be None
        use_alt = True
    else:
        km = m["distance"]
        use_alt = False
    if km is None:
        return {"feasible": False, "why": "route"}
    fuel = math.ceil(km / 10)
    time = math.ceil(km * WEATHER[weather])
    if time > m["deadline"]:
        return {"feasible": False, "why": "deadline"}
    if (fuel > resources["fuel"] or m["kits"] > resources["kits"]
            or m["personnel"] > resources["personnel"]):
        return {"feasible": False, "why": "resources"}
    return {"feasible": True, "fuel": fuel, "alt": use_alt}
```

The **why** field matters: in Round 3, a failure caused by resources and a failure caused by route or deadline lead to different decisions.

Round 1: Basic Rescue

- **calculate_fuel:** ceil(distance / 10).
- **calculate_travel_time:** ceil(distance x 1.0).
- **assess_mission:** check fuel, kits, personnel and deadline.
- **solve_round1:** if feasible, dispatch (priority = severity, mission_id, fuel_needed); otherwise reject or wait with a reason such as 'Insufficient fuel'.

Round 2: Blocked Routes

- Blocked and alternate is null: infeasible, reject.

- Blocked and an alternate exists: compute fuel and time from the alternate distance.
- If feasible, action = reroute and uses_alternate_route = True.
- With two missions, take the feasible one with higher severity.
- Include mission_id when there is more than one mission.

Round 3: Limited Resources

- Run check_mission on every mission.
- If any are feasible, dispatch or reroute the best one.
- If none is feasible: only a resource shortage and resupply_possible = True gives **resupply**; an impossible route or deadline gives **reject**; anything else gives **wait**.
- Never send the most severe mission if resources cannot support it.

Round 4: Critical Priorities

- Remove infeasible missions.
- Sort the rest by severity (higher first), deadline (tighter first), fuel (lower first).
- Pick the first; priority = its severity.
- If none is feasible, use the Round 3 fallback.

```
best = min(feasible_list, key=lambda x: (-x["severity"], x["deadline"], x["fuel"]))
```

This one line handles all three tie-breaks and keeps the result deterministic.

Round 5: Adaptive

- Read case["update_report"].
- **Call Ollama** (mandatory) to extract the new weather, blocked missions and new resources.
- Verify the LLM's answer: is the weather valid, does the mission id exist, is the fuel a number?
- Update the case, for example case["weather"], route_blocked = True, resources["fuel"] = 4.
- Run the Round 4 logic on the updated case and return the decision.
- Fallback: if the LLM fails, parse the report with a regex (re). Note that a successful Ollama call is still required for every Round 5 case.

call_ollama(prompt): use requests.post to OLLAMA_URL, with an Authorization: Bearer header and a body containing model and messages. Return None on any error. The app's --test-api option tests exactly this function.

Local testing (app.py)

```
python app.py                          # all sample cases
python app.py --round 3                # one round only
python app.py --case r2_p1 --show      # one case, with input and output
python app.py --test-api               # test the Ollama setup
```

app.py only checks format and safety. It does not give the official competition score.

Final checklist

- Output always contains action, priority (integer 1-5) and reason (non-empty).
- Never use a blocked road.
- Always compute fuel from the route actually used.
- Rounds 1-4 must work without an LLM.

- Never put the API key in code; use only the environment variables OLLAMA_API_KEY and OLLAMA_MODEL.
- Do not submit API keys, .env, .venv or __pycache__.
- Test in order: Round 1, 2, 3, 4, then 5.
- Submit round by round, starting from Round 1, on the organizer's page.